

Preprocessing: documents to paragraph units

```
tx <- list(
  seed = 2026L,
  min_anchor = 30L, # author paragraphs needed to anchor the target length
  window_n = 300L, # embedding requests per quota window (one per document)
  window_wait = 4L * 60L * 60L, # seconds to sleep at the window boundary
  use_openrouter = TRUE, # FALSE to use the OpenAI-compatible endpoint
  # base_url_env and api_key_env are the NAMES of .Renv variables, so
  # nothing sensitive appears here. Ignored when use_openrouter = TRUE.
  steps = tibble::tribble(
    ~step, ~base_url_env, ~api_key_env, ~model,
    "embed", "COMPASS_EMBED_URL", "COMPASS_EMBED_KEY", "MODEL-FOR-EMBEDDING")
)

suppressPackageStartupMessages({
  library(here); library(viridis)
  library(tidyverse) # tidyverse LAST so its verbs win
})
# Used namespaces: ragnar (embeddings), glue, quanteda (default corpus only).
theme_set(theme_minimal(base_size = 12))
options(knitr.table.max.print = Inf, knitr.kable.max_rows = Inf,
  max.print = 1e6)
set.seed(tx$seed)
`%||` <- function(x, y) if (is.null(x)) y else x
p_out <- function(f) here::here("outputs", f)
source(here::here("code", "llm_source.R"))
if (isTRUE(tx$use_openrouter)) {
  source(here::here("code", "llm_openrouter.R"))
  tx$steps$model <- open_router_models[tx$steps$step]
}
dir.create(here::here("outputs"), showWarnings = FALSE, recursive = TRUE)
```

Corpus

The default corpus is `quanteda::data_corpus_inaugural`: 60 US inaugural addresses, 1789 to 2025, with Year already carried as a docvar. It ships with `quanteda`, so there is no extra dependency, and it is a real corpus of multi-paragraph documents rather than a simulation, so the linkage runs against text that behaves like text.

Two properties worth knowing before you read anything into the results. It yields about 1,874 paragraphs, roughly a tenth of a production corpus, so K in `02_topics.qmd` belongs in the low teens. And it has **one document per year**, so document clusters and year strata coincide; that

is fine for preprocessing and topic modeling, but the cluster-robust variance in 03_stance.qmd cannot be exercised on it.

For your own data, replace this chunk with the read of your own table. Everything downstream is unchanged; the validation gate below runs on whatever filled corpus_raw.

```
corpus_raw <- tibble(
  atom_id = quanteda::docnames(quanteda::data_corpus_inaugural),
  Year = quanteda::docvars(quanteda::data_corpus_inaugural, "Year"),
  body_english = as.character(quanteda::data_corpus_inaugural))

# For your own data, this chunk becomes one line:
# corpus_raw <- readr::read_csv(here("input", "YOUR_FILE.csv"))
# Required columns: atom_id (document id), Year (integer), body_english (text).
```

Corpus validation (keep this at work)

Stops the render with a named error on a bad read, which is cheaper than debugging a silent failure five chunks later.

```
miss <- setdiff(c("atom_id", "Year", "body_english"), names(corpus_raw))
if (length(miss)) stop("corpus_raw missing column(s): ",
  str_c(miss, collapse = ", "))
stopifnot(
  "atom_id must be unique" = !anyDuplicated(corpus_raw$atom_id),
  "Year must be integer-valued" = all(corpus_raw$Year ==
    as.integer(corpus_raw$Year)),
  "body_english must be character" = is.character(corpus_raw$body_english))
n0 <- nrow(corpus_raw)
corpus_raw <- corpus_raw |>
  dplyr::filter(!is.na(Year), !is.na(body_english),
    str_squish(body_english) != "")
glue::glue("Validation: kept {nrow(corpus_raw)} of {n0} comments, ",
  "{min(corpus_raw$Year)}-{max(corpus_raw$Year)}")
```

Validation: kept 60 of 60 comments, 1789-2025

Paragraph linkage

Documents rarely arrive with clean paragraphs. Some authors write real paragraphs; others put a blank line after every sentence, and splitting on blank lines then yields orphan sentences too short for a topic model, which is where the STM falls apart. This section rebuilds units of a defensible length without assuming either convention and without a hand-picked size.

Blocks are the text between newline runs. **Which documents need linkage** is decided per document: a document is *unparagraphed* when its median block is a single sentence, meaning

the author broke after nearly every sentence. Paragraphed documents pass through untouched, because their blocks already are the author's paragraphs. That rule has no tuned threshold.

The target length comes from the corpus. It is the median length of all paragraphs in the paragraphed documents: how long this corpus's authors make a paragraph when they do make one. If fewer than 30 such paragraphs exist, the render stops by name rather than falling back to an arbitrary number; that guard protects the stability of the median and is not a tuning knob.

Linkage is agglomerative with a contiguity constraint, the PSU-formation move of combining small units with a neighbor until a minimum measure of size is met. Within each unparagraphed document, among adjacent pairs where at least one block is below the floor, the most semantically similar pair merges first, repeating until every unit reaches the floor. Orphans link to each other when they belong together, a heading joins the paragraph it introduces because it shares its vocabulary, and a block reaches across to a dissimilar neighbor only when nothing better remains. Reading order is never broken. The procedure is deterministic, so no random start is involved: it defines the document unit, and a rerun must reproduce it exactly.

The floor is calibrated, not chosen. Because merging stops once a unit reaches the floor, linked units land between one and two floors, so the output median sits above the floor. The floor is therefore found by search: the value at which the median length of relinked units matches the target. The whole method reduces to one statement a reader can check: *unparagraphed documents were relinked into units whose median length matches the median length of author-formed paragraphs in the same corpus.*

Similarity is cosine on embeddings of the raw block text. Only unparagraphed documents are embedded, one request per document, and vectors are cached so nothing is sent twice. After a merge, the unit's vector is the token-weighted mean of its parts, an approximation to re-embedding the merged text adopted because it costs no extra requests. Calibration and the sensitivity test reuse the cached vectors, so neither makes an API call.

```
blocks <- split_blocks(corpus_raw)
plan <- plan_linkage(blocks, tx$min_anchor)
n_unpar <- sum(!plan$doc$paragraphed)
writeLines(glue::glue("{nrow(blocks)} blocks from {nrow(plan$doc)} documents. \
\
{sum(plan$doc$paragraphed)} documents have paragraph markup and pass through
\\
untouched; {n_unpar} are unparagraphed and will be relinked. Target length =
\\
{plan$target} tokens (median of {plan$n_anchor} author paragraphs; \
interquartile range {plan$quartiles[1]} to {plan$quartiles[2]})."))
```

1874 blocks from 60 documents. 54 documents have paragraph markup and pass through untouched; 6 are unparagraphed and will be relinked. Target length = 70 tokens (median of 1345 author paragraphs; interquartile range 40 to 121).

```
writelnLines(glue::glue("Embedding requests needed: {n_unpar} (one per \\
  unparagraphed document), about {ceiling(n_unpar / tx$window_n)} quota \\
  window(s)."))
```

Embedding requests needed: 6 (one per unparagraphed document), about 1 quota window(s).

```
unpar_blocks <- blocks |>
  dplyr::semi_join(dplyr::filter(plan$doc, !paragraphed), by = "atom_id")
emb <- if (nrow(unpar_blocks) == 0) list(vec = list()) else
  embed_census(unpar_blocks, p_out("block_embeddings.rds"), tx$steps,
    tx$use_openrouter,
    tx$window_n, tx$window_wait)
```

```
cal <- calibrate_floor(blocks, emb$vec, plan$doc, plan$target)
paragraphs <- link_corpus(blocks, emb$vec, cal$floor, plan$doc) |>
  dplyr::select(para_uid, atom_id, para_id, Year, n_tokens, text, relinked,
    n_blocks, merge_sim)

rl <- dplyr::filter(paragraphs, relinked)
sims <- unlist(rl$merge_sim)
tibble::tibble(
  measure = c("blocks in", "paragraphs out", "calibrated floor (tokens)",
    "target: median author paragraph", "median of relinked units",
    "relinked units", "merges performed (blocks absorbed)",
    "relinked units below floor (whole short documents)",
    "merge similarity, median", "merge similarity, 10th percentile"),
  value = c(nrow(blocks), nrow(paragraphs), round(cal$floor, 1), plan$target,
    stats::median(rl$n_tokens), nrow(rl), sum(rl$n_blocks - 1L),
    sum(rl$n_tokens < cal$floor),
    round(stats::median(sims), 3),
    round(stats::quantile(sims, .1, names = FALSE), 3))) |>
  tbl_paged(caption = "Linkage diagnostics. The median of relinked units should
  sit near the target. A low 10th-percentile merge similarity means some units
  joined a dissimilar neighbor because nothing better remained; read those before
  trusting the parse.")
```

Linkage diagnostics. The median of relinked units should sit near the target. A low 10th-percentile merge similarity means some units joined a dissimilar neighbor because nothing better remained; read those before trusting the parse.

measure	value
blocks in	1874.000
paragraphs out	1484.000
calibrated floor (tokens)	36.000

measure	value
target: median author paragraph	70.000
median of relinked units	67.000
relinked units	139.000
merges performed (blocks absorbed)	390.000
relinked units below floor (whole short documents)	0.000
merge similarity, median	0.842
merge similarity, 10th percentile	0.801

Handoff

Everything the downstream scripts need is written here, and nothing downstream re-runs this script's work. paragraphs.csv is the corpus for 02_topics.qmd. blocks.csv and linkage_plan.rds let script 2 relink at other floors for the sensitivity test using the cached vectors, with no API calls.

```
paragraphs |>
  dplyr::count(Year) |>
  ggplot(aes(Year, n)) +
  geom_col(fill = viridis(1, begin = 0.35)) +
  labs(x = NULL, y = "paragraph units")
```

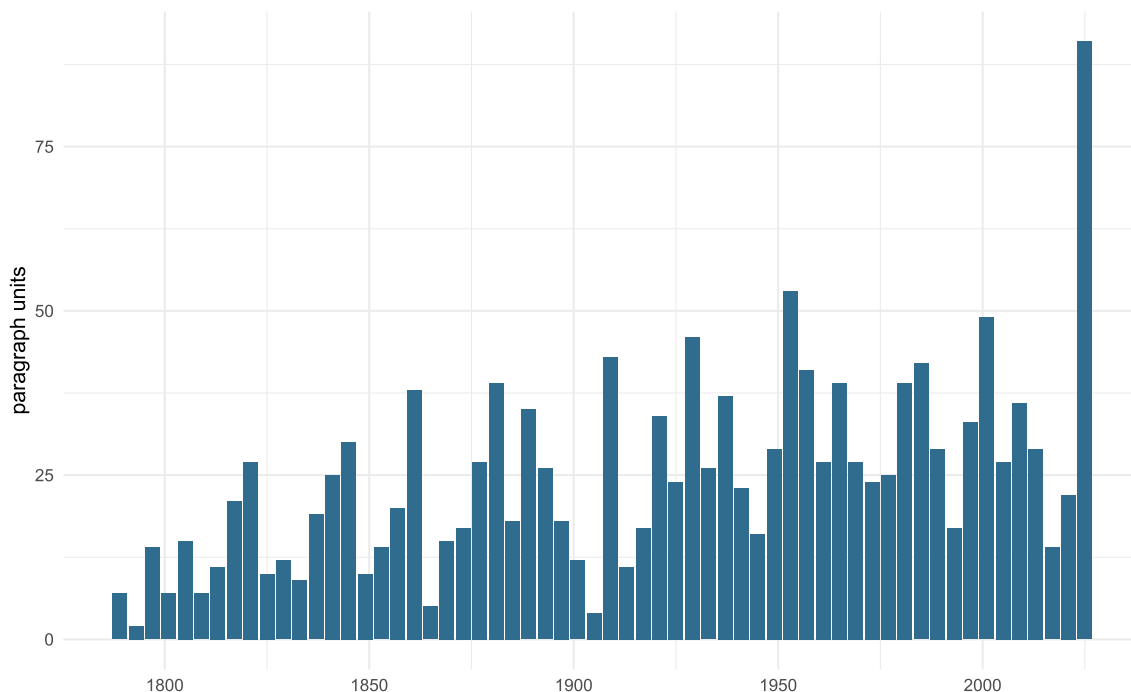


Figure 1: Paragraph units by year: the denominator for every time-indexed estimate downstream.
Thin years mean trend splines are interpolating there.

```
readr::write_csv(dplyr::select(paragraphs, -merge_sim),  
p_out("paragraphs.csv"))  
readr::write_csv(blocks, p_out("blocks.csv"))  
saveRDS(list(doc = plan$doc, target = plan$target, n_anchor = plan$n_anchor,  
floor = cal$floor), p_out("linkage_plan.rds"))  
writeLines(glue::glue("Wrote paragraphs.csv ({nrow(paragraphs)} units), \\  
blocks.csv ({nrow(blocks)} blocks), linkage_plan.rds (floor \\  
{round(cal$floor, 1)}, target {plan$target}), block_embeddings.rds. \\  
Next: 02_topics.qmd."))
```

Wrote paragraphs.csv (1484 units), blocks.csv (1874 blocks), linkage_plan.rds (floor 36, target 70),
block_embeddings.rds. Next: 02_topics.qmd.